

When displaying geospatial data, sometimes you want to be able to enable your user to see this data in relation to geographic elements- from roads to cities and towns and geographic boundaries- such as what you see in Google maps- or in relation to the environmental context, from aerial images. Ideally, you want to pan (move right, left, up and down) and zoom on these maps. As you zoom in, the map API should update the content that is fed to the browser, to show more details about a smaller area.

We can do this with the help of map APIs. Map API's will use a system of tiles, configured in a pyramid, with the top tile representing the view 'zoomed out' fully, and each step representing a deep level of zoom. The tiles cover increasing portions of the whole map. This is the approach behind Google maps and other tools.



We are going to use Leaflet in conjunction with d3 to display data on a map, in its spatial context. Leaflet is a lightweight Javascript library for interactive maps. It is free to use, and works on major browsers. It can be extended with plugins and extensions, and many developers work with it. Info here: leafletjs.com

Leaflet will do the work of updating the map as you pan and zoom, d3 will display the data in a layer situated on top of the Leaflet map, and there will be a translation step between 'pixel space' and latitude and longitude coordinates using a function provided by Leaflet, which considers the current state of the map.

How can we integrate Leaflet with d3?

First, we need to **download leaflet and include its js and css files in our project directory structure.**

Our project structure will look familiar: **index.html** (for html elements), **a css folder** for style settings, a **js folder** which includes **main.js** and our new map javascript class **'leafletMap.js'**. This is different than leaflet.js- which is the map library. I wrote leafletMap.js.

In our html file, we need to create a container (a div) for the map, such as:

```
<div id="my-map"></div>
```

We can set the size of the map container in css, or elsewhere

```
#my-map { width: 100%; height: 500px; }
```

This means the map will fill the width of the page.

In main, we load the data, in this case we have a file with global cities, including a city name and population. This is the same loading and basic processing we have seen before.

```
d3.csv('data/worldcities.csv')

.then(data => {

  data.forEach(d => {

    d.latitude = +d.lat; //make sure these are not strings

    d.longitude = +d.lng; //make sure these are not strings

  });

  console.log(data); //ok, got my data!
```

```
// Initialize chart and then show it

leafletMap = new LeafletMap({ parentElement: '#my-map'}, data);

})

.catch(error => console.error(error));
```



Here you could also add in additional views of your data, in their own class instances, created here.

Then, when initializing our Leaflet map, we need to create the map through several steps. We need a url to a map tile server, which will feed us the map tiles at the appropriate pan coordinates and zoom level, when the user navigates. We put this into a layer within Leaflet.

```
//ESRI

vis.esriUrl =
'https://server.arcgisonline.com/ArcGIS/rest/services/World_Imagery/MapServer
/tile/{z}/{y}/{x}';

vis.esriAttr = 'Tiles &copy; Esri &mdash; Source: Esri, i-cubed, USDA,
USGS, AEX, GeoEye, Getmapping, Aerogrid, IGN, IGP, UPR-EGP, and the GIS User
Community';

//this is the base map layer, where we are showing the map background

vis.base_layer = L.tileLayer(vis.esriUrl, {
```

```

    id: 'esri-image',
    attribution: vis.esriAttr,
    ext: 'png'
  });

  vis.theMap = L.map('my-map', {
    center: [30, 0],
    zoom: 2,
    layers: [vis.base_layer]
  });

```

At this point, we would have a map that looks like this:

Now, how do we get data onto the map? Leaflet supplies tools to add vector graphic elements, shapes, text, tooltips onto the map, but I first want to show you how to do this with a d3 layer, that coordinates with Leaflet.

The key step is passing off the placement of elements to Leaflet, rather than doing our usual approach of setting a scale and mapping the data to a position in space. Leaflet is going to take care of going from latitude and longitude, to a coordinate on the screen, taking into account the zoom level and the position of your bounding box within the whole map.

```

//initialize svg for d3 to add to map

  L.svg({clickable:true}).addTo(vis.theMap)// we have to make the svg layer
clickable

  vis.overlay = d3.select(vis.theMap.getPanes().overlayPane)
  vis.svg = vis.overlay.select('svg').attr("pointer-events", "auto")
  //these are the city locations, displayed as a set of dots
  vis.Dots = vis.svg.selectAll('circle')
    .data(vis.data)

```

```

        .join('circle')

        .attr("fill", "steelblue")

        .attr("stroke", "black")

//Leaflet has to take control of projecting points. Here we are feeding the
latitude and longitude coordinates to

//leaflet so that it can project them on the coordinates of the view. Notice,
we have to reverse lat and lon.

        //Finally, the returned conversion produces an x and
y point. We have to select the the desired one using .x or .y

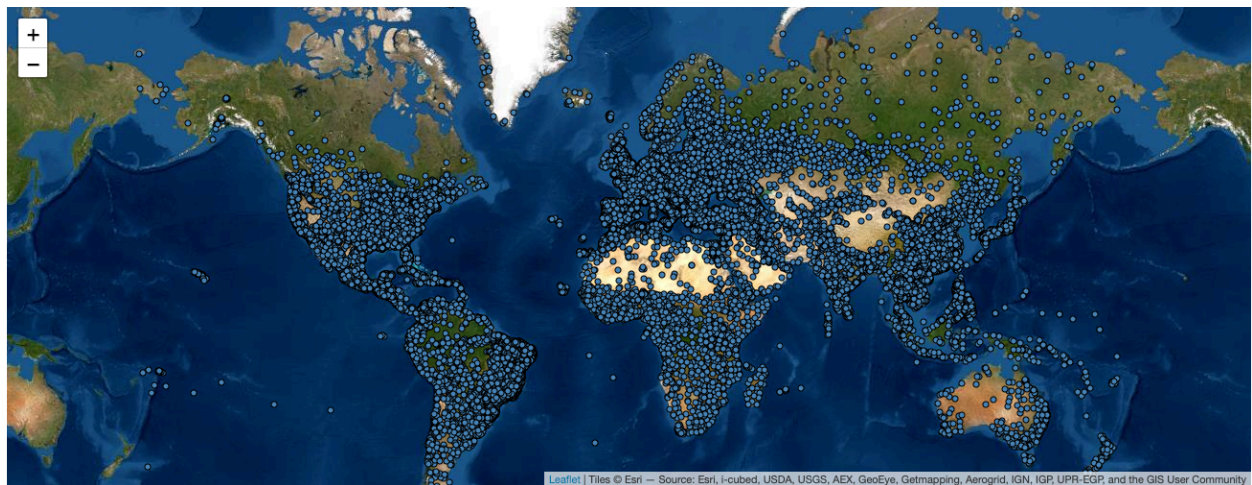
        .attr("cx", d =>
vis.theMap.latLngToLayerPoint([d.latitude,d.longitude]).x)

        .attr("cy", d =>
vis.theMap.latLngToLayerPoint([d.latitude,d.longitude]).y)

        .attr("r", 3);

```

And here's what this looks like:



Not too bad huh?

When you zoom, you need to have an event handler to update the positions of the points. You can attach it to the map:

```

//handler here for updating the map, as you zoom in and out

vis.theMap.on("zoomend", function(){

    vis.updateVis();

```

```
});
```

Then, your update vis function needs to reposition the points, like so:

```
updateVis() {  
  let vis = this;  
  
  //redraw based on new zoom- need to recalculate on-screen position  
  
  vis.Dots  
    .attr("cx", d =>  
vis.theMap.latLngToLayerPoint([d.latitude,d.longitude]).x)  
    .attr("cy", d =>  
vis.theMap.latLngToLayerPoint([d.latitude,d.longitude]).y)  
    .attr("r", 3) ;//keeping the radius the same, but you could change it...  
}
```

You can add tooltips or interactive features, by attaching event handlers to each of these points, see the code in the link for an example of this.

It isn't as straightforward as working directly with d3 alone. For instance, I tried to inspect the html elements, to see the svg where my points were added, and it took me a minute to find them (in the leaflet overlay pane, which is the top-most layer in the leaflet map).

```
▼<div id="my-map" class="leaflet-container leaflet-touch leaflet-fade-anim leaflet-grab leaflet-touch-drag leaflet-touch-zoom" tabindex="0" style="position: relative;">  
  ▼<div class="leaflet-pane leaflet-map-pane" style="transform: translate3d(0px, 0px, 0px);">  
    ▶<div class="leaflet-pane leaflet-tile-pane">_</div>  
    <div class="leaflet-pane leaflet-shadow-pane"></div>  
    ▼<div class="leaflet-pane leaflet-overlay-pane"> = $0  
      ▼<svg pointer-events="auto" class="leaflet-zoom-animated" width="1559" height="600" viewBox="-130 -50 1559 600" style="transform: translate3d(-130px, -50px, 0px);">  
        <g></g>  
        <circle fill="steelblue" stroke="black" cx="1046" cy="231" r="3"></circle>  
        <circle fill="steelblue" stroke="black" cx="953" cy="358" r="3"></circle>  
        <circle fill="steelblue" stroke="black" cx="869" cy="255" r="3"></circle>  
        <circle fill="steelblue" stroke="black" cx="856" cy="285" r="3"></circle>  
        <circle fill="steelblue" stroke="black" cx="993" cy="298" r="3"></circle>  
        <circle fill="steelblue" stroke="black" cx="995" cy="247" r="3"></circle>  
        <circle fill="steelblue" stroke="black" cx="516" cy="409" r="3"></circle>  
        <circle fill="steelblue" stroke="black" cx="1010" cy="225" r="3"></circle>
```

But otherwise this is d3. You can create color scales, you can control attributes and styles based on data, and you can add event handlers and transitions, as usual. The only difference- if you need to position or size elements based on map zoom and bounding box position, you will need the help of leaflet- and if you add new elements, make sure you add them to this leaflet-overlay-pane layer, or a new layer you create in the map.

New backgrounds:

What if you want a different map background? You can swap them relatively easily. Look here: <https://leaflet-extras.github.io/leaflet-providers/preview/>

Think about which map backgrounds will support users interested in understanding the spatial context for the collection of plant specimens.

Using Leaflet elements instead of d3:

Leaflet has shapes you can add to maps, using Leaflet functions. Given the integration with d3 functionality in the larger project, I suggest sticking with d3 for plotting elements.